

Tema 1 — Redes Neuronales, Deep Learning y Backpropagation

1. PERCEPTRÓN SIMPLE

Una neurona artificial que recibe entradas, las multiplica por pesos, suma todo (incluyendo un bias/umbral), y pasa el resultado por una función de activación para producir una salida.

Fórmula:

$$\text{salida} = \text{activación}(x_1 * w_1 + x_2 * w_2 + \dots + x_n * w_n + b)$$

Entrenamiento: Se inicializan los pesos de forma aleatoria. Se compara la salida con el valor esperado, se calcula el error, y se ajustan los pesos multiplicando el error por una tasa de aprendizaje (α , valor entre 0 y 1). Se repite hasta que el error sea aceptable.

Función escalón (Heaviside): La función de activación más básica. Devuelve 1 si la entrada es ≥ 0 , y 0 en caso contrario.

Limitación clave: El perceptrón simple SOLO puede resolver problemas linealmente separables. NO puede resolver el problema XOR, porque no existe una única línea recta que separe las clases. Para resolver XOR se necesita un perceptrón multicapa.

2. PERCEPTRÓN MULTICAPA (MLP / Fully Connected)

Es una red con al menos una capa oculta entre la entrada y la salida. Cada neurona de una capa está conectada a todas las neuronas de la siguiente capa (por eso "fully connected").

Estructura:

Capa de entrada (i) $\rightarrow$ Capa(s) oculta(s) (h) $\rightarrow$ Capa de salida (o)

Cada conexión tiene un peso (w). Cada capa tiene un bias (b). Los pesos se inicializan aleatoriamente y se ajustan durante el entrenamiento con backpropagation.

¿Cómo se calcula la salida? (Forward pass / propagación hacia adelante)

1. Para cada neurona oculta h : calcular la entrada neta $= \sum(x_i * w_i) + b$
 2. Aplicar la función de activación a esa entrada neta $\rightarrow$ salida de h
 3. Usar las salidas de h como entradas de la capa siguiente
 4. Repetir hasta la capa de salida
-

3. BACKPROPAGATION

¿Qué es? Es el método utilizado para entrenar redes neuronales, ajustando los pesos de la red para minimizar el error entre la salida predicha y la salida real.

¿Qué tipo de método es? Es un método de optimización supervisado (necesita datos etiquetados). Utiliza el cálculo de derivadas parciales y la regla de la cadena para propagar el error desde la capa de salida hacia las capas anteriores.

¿Cómo funciona paso a paso?

1. **Forward pass:** Se pasan los datos por la red y se obtiene una predicción.
2. **Cálculo del error:** Se compara la predicción con el valor real usando una función de error (por ejemplo, error cuadrático medio):

$$E = \frac{1}{2} * (\text{valor_real} - \text{valor_predicho})^2$$

3. **Backward pass (la parte "back"):** Se calcula cuánto contribuyó cada peso al error, usando la regla de la cadena:

$$\partial E / \partial w_5 = \partial E / \partial \text{salida} * \partial \text{salida} / \partial \text{entrada_neta} * \partial \text{entrada_neta} / \partial w_5$$

Esto se hace desde la última capa hacia la primera (de ahí "back" propagation — se propaga el error hacia atrás).

4. **Actualización de pesos:** Cada peso se ajusta en la dirección que reduce el error:

$$w_{\text{nuevo}} = w_{\text{viejo}} - \alpha * \partial E / \partial w$$

Donde α es la tasa de aprendizaje (learning rate).

5. Se repiten los pasos 1-4 muchas veces (epochs) hasta que el error sea suficientemente bajo.

Dato clave para el final: Backpropagation NO es un método alternativo al descenso por gradiente. Es una forma de APLICAR el descenso por gradiente a redes neuronales. Es decir, backpropagation USA gradient descent.

4. FUNCIONES DE ACTIVACIÓN

Las funciones de activación introducen no-linealidad en la red. Sin ellas, no importa cuántas capas tenga la red: seguiría siendo equivalente a una transformación lineal.

Para backpropagation, las funciones de activación DEBEN ser derivables (porque se usa el gradiente para actualizar los pesos).

Sigmoide

- Fórmula: $\sigma(x) = 1 / (1 + e^{(-x)})$
- Rango de salida: entre 0 y 1
- Uso: clasificación binaria (capa de salida), capas ocultas en redes pequeñas
- Problema: desvanecimiento del gradiente en valores muy altos o muy bajos

ReLU (Rectified Linear Unit)

- Fórmula: $f(x) = \max(0, x)$
- Rango de salida: 0 a infinito
- Uso: capas ocultas (especialmente en CNN)
- Ventaja: rápida de calcular, no sufre tanto desvanecimiento del gradiente
- Problema: "neuronas muertas" (si la entrada es negativa, la salida siempre es 0)

Softmax

- Toma un vector de valores numéricos (logits) y los transforma en una distribución de probabilidad (cada valor entre 0 y 1, todos suman 1)
- Uso: capa de salida en clasificación multiclase con clases excluyentes
- Permite interpretar la salida como la probabilidad/confianza del modelo de que la entrada pertenezca a cada clase

¿Qué función usar según el problema?

Tipo de problema	Función de activación en la última capa
Regresión	Lineal, Sigmoide, ReLU
Clasificación binaria	Sigmoide
Clasificación multiclase (clases excluyentes)	Softmax

5. MÉTODOS DE REGULARIZACIÓN (para evitar overfitting)

Son técnicas que ayudan a que el modelo generalice mejor, es decir, que funcione bien con datos que nunca vio.

Regularización L1 y L2

Penalizan el valor de los pesos de la red, evitando que se le dé demasiada relevancia a una

característica sobre otra.

- **L1 (Lasso):** Agrega a la función de pérdida la suma de los valores absolutos de los pesos: $\lambda * \sum |w|$. Tiende a llevar pesos a exactamente 0 (selección de features).
- **L2 (Ridge):** Agrega a la función de pérdida la suma de los cuadrados de los pesos: $\lambda * \sum w^2$. Tiende a hacer los pesos pequeños pero no cero.

Dato clave: L1 y L2 NO reemplazan backpropagation. Modifican la función de pérdida, pero backpropagation sigue siendo el mecanismo para calcular los gradientes y actualizar los pesos. Trabajan juntos.

Dropout

Método de regularización que "apaga" (pone en 0) neuronas aleatoriamente durante el entrenamiento. Esto fuerza a la red a no depender de unas pocas neuronas y a aprender de forma más robusta.

- Durante entrenamiento: se apagan neuronas al azar en cada iteración
- Durante predicción/test: se usan todas las neuronas (no se apaga ninguna)

Early Stopping

La idea es detener el entrenamiento antes de que el error del conjunto de validación empiece a aumentar. Se monitorea el error de validación en cada epoch:

- Si el error de validación sigue bajando → seguir entrenando
- Si el error de validación empieza a subir → PARAR, la red está empezando a sobreajustar

Se queda con los pesos de la iteración donde el error de validación era mínimo.

Data Augmentation (Aumento de datos)

Genera datos nuevos a partir de los existentes aplicando transformaciones: rotaciones, recortes, inversiones, cambios de brillo, etc.

- Especialmente útil cuando se trabaja con imágenes
- Aumenta el tamaño efectivo del dataset sin recolectar datos nuevos

6. OPTIMIZADORES

Un optimizador es una implementación concreta del algoritmo de backpropagation. Todos buscan minimizar la función de pérdida ajustando los pesos, pero cada uno tiene una estrategia diferente.

SGD (Stochastic Gradient Descent)

- Backpropagation básico, sin optimización
- Actualiza los pesos usando el gradiente calculado sobre un mini-batch aleatorio de datos (no todo el dataset)
- Problema: puede ser lento y oscilar mucho

Momentum

- El gradiente se usa para la aceleración, no para la velocidad
- Acumula una "velocidad" de las actualizaciones anteriores, como una bola rodando cuesta abajo que acumula inercia
- Ayuda a superar mínimos locales y acelera la convergencia

Nesterov

- Variante de Momentum: en vez de calcular el gradiente en el punto actual, lo calcula en el punto donde el momentum lo llevaría
- Suele converger un poco más rápido que Momentum estándar

AdaGrad

- Adapta la tasa de aprendizaje para cada parámetro individualmente
- Reduce el learning rate en las dimensiones más empujadas
- Problema: el learning rate puede volverse demasiado pequeño con el tiempo

RMSPProp

- Soluciona el problema de AdaGrad: usa un promedio móvil exponencial de los gradientes al cuadrado en vez de acumularlos todos
- Funciona bien en problemas no estacionarios

Adam (Adaptive Moment Estimation)

- Combina las ideas de Momentum + RMSPProp
- Adapta la tasa de aprendizaje por parámetro Y usa momentum
- Es el optimizador más usado en la práctica
- Generalmente converge más rápido que RMSPProp y Momentum por separado

AdaMax

- Modificación de Adam que usa la norma infinita en vez de L2

Nadam

- Adam + Nesterov: suele converger más rápido que Adam

AdaDelta

- Variación de AdaGrad que usa promedios móviles

Resumen rápido para el final: Si te preguntan cuál es el más rápido/usado, la respuesta es Adam. Si te preguntan la relación entre ellos: Adam = Momentum + RMSProp.

7. REDES CONVOLUCIONALES (CNN)

Dominan la visión por computadora. Fueron desarrolladas por Yann LeCun. La idea central es que los filtros (kernels) detectan patrones espaciales en las imágenes (bordes, texturas, formas) de forma jerárquica.

La arquitectura se divide en dos grandes etapas:

- **Feature Learning (aprendizaje de características):** Las capas convolucionales + ReLU + Pooling. Extraen patrones de la imagen.
- **Classification (clasificación):** La capa Fully Connected al final. Usa las features extraídas para clasificar.

Arquitectura típica completa

El esquema que piden dibujar en el final es este orden:

```
ENTRADA (imagen)
  ↓
[Capa Convolucional] → aplica filtros que detectan patrones
  ↓
[Capa ReLU] → función de activación (introduce no-linealidad)
  ↓
[Capa Pooling] → reduce la dimensión espacial (ej: Max Pooling)
  ↓
  ↓ ← Se repite N veces: Conv → ReLU → Pooling
  ↓
[Capa Fully Connected (FC)] → clasifica (al final, solo 1 capa FC)
  ↓
SALIDA (clase predicha)
```

Nota de las diapos: "Múltiples capas Convolucional, RELU y Pooling, pero solo 1 FC" (slide 32 de la presentación de Deep Learning).

Capa Softmax al final: Si el problema es de clasificación multiclase, se suele agregar una capa Softmax después de la Fully Connected para asignar probabilidades a cada clase.

¿Qué hace cada capa?

Capa Convolutiva:

- Aplica filtros (matrices pequeñas, ej: 3x3) sobre la imagen
- Cada filtro se "desliza" por la imagen calculando el producto punto
- Detecta patrones: los primeros filtros detectan bordes simples, los más profundos detectan formas complejas (ojos, ruedas, etc.)
- Esto es la operación de CONVOLUCIÓN: combinar dos funciones (el filtro y la región de la imagen) en un valor

Capa ReLU:

- Aplica la función ReLU: $f(x) = \max(0, x)$
- Reemplaza todos los valores negativos por 0
- Se entrena con backpropagation

Capa Pooling:

- Reduce la dimensión espacial de la representación
- Max Pooling: toma el valor máximo de una región (ej: de cada 2x2 toma el más alto)
- Reduce la cantidad de parámetros y el cómputo
- Hace la detección de features más robusta a pequeños desplazamientos

Capa Fully Connected (FC):

- Al final de la red, después de varias rondas de Conv+ReLU+Pooling
 - Toma todas las features extraídas y las usa para clasificar
 - Es como un perceptrón multicapa tradicional
-

8. REDES RECURRENTES (RNN)

Útiles cuando los patrones en los datos cambian con el tiempo (secuencias). A diferencia de las redes feedforward, las RNN tienen conexiones que forman ciclos: la salida de un paso temporal se usa como entrada del siguiente.

Celdas de memoria y variables ocultas

La parte de la red que mantiene estado a lo largo de los pasos de tiempo. La salida en el paso t depende de la entrada actual Y y de todas las salidas anteriores.

Variables ocultas: Son el estado interno (memoria) que la RNN va llevando a lo largo de los pasos de tiempo. No aparecen como dato observado, pero determinan lo que la red hace en cada instante. En el tiempo t , la red usa la entrada $X(t)$ y también la salida anterior $Y(t-1)$, lo que hace que la salida actual dependa de lo anterior — por eso se dice que tiene "memoria".

3 tipos de estructuras

Estructura	Entrada	Salida	Ejemplo
Secuencia → Secuencia	Serie temporal	Serie temporal	Predicción de precios de acciones (N días → N días desplazados)
Secuencia → Valor singular	Secuencia de palabras	Un valor	Clasificación de sentimiento de una reseña (palabras → positivo/negativo)
Valor singular → Secuencia	Una imagen	Secuencia de palabras	Generación de subtítulos para imágenes (image captioning)

Problema principal: desvanecimiento del gradiente

En RNNs, el problema del desvanecimiento del gradiente es **exponencialmente peor** que en redes feedforward, porque 1 capa recurrente con 100 pasos temporales equivale a entrenar una red de 100 capas hacia adelante.

Soluciones al desvanecimiento del gradiente

- **Gating:** Técnica que le dice a la red cuándo olvidar la entrada anterior y cuándo recordarla
- **LSTM (Long Short-Term Memory):** Usa compuertas (forget gate, input gate, output gate) para controlar qué información se mantiene y cuál se descarta. Mejor para dependencias largas pero más pesada (más parámetros, más cómputo).
- **GRU (Gated Recurrent Unit):** Versión simplificada de LSTM con menos parámetros. Más simple y fácil de ajustar, pero puede no capturar dependencias tan largas como LSTM.
- **Gradient Clipping:** Limita el valor del gradiente para que no explote ni desaparezca

Perplejidad (métrica de RNN)

Métrica usada para evaluar modelos de lenguaje. Mide qué tan "convencido" está el modelo de que adivinará la palabra siguiente. Está relacionada con la entropía y la entropía cruzada.

- Cuanto MENOR sea la perplejidad, MEJOR es el modelo
- Valores de referencia (depende del contexto):
 - Menos de 100: buena
 - Entre 100 y 300: aceptable

- Más de 300: mala

RNN vs Red feedforward

- Red feedforward → clasificación o regresión (datos estáticos)
- Red recurrente → series temporales, predicción, pronóstico (datos secuenciales)

Redes Neuronales de Tensores Recursivas (RNTN)

No confundir con las recurrentes. Son redes con estructura de árbol (no secuencial) diseñadas para análisis de sentimiento en textos.

- Tienen en cuenta el orden y la agrupación sintáctica de las palabras
 - Tienen neuronas raíz y neuronas hojas
 - El nodo raíz genera un "score" que se reingresa recursivamente en los nodos hojas
 - Se entrenan con backpropagation, comparando el árbol de dependencias obtenido con uno generado manualmente
 - Usos: análisis de sentimientos, detección de componentes en imágenes
-

9. EMBEDDINGS DE PALABRAS (Word Embeddings)

¿Qué son?

Mapean palabras a **vectores de números reales densos**, donde la cercanía geométrica entre dos vectores refleja similitud semántica entre esas palabras. Se basan en la **hipótesis distribucional**: "dime con quién andas y te diré qué significas" — es decir, palabras que aparecen en contextos similares tienden a tener significados similares.

Ejemplo de la idea: en el espacio de embeddings, el vector de "rey" menos el vector de "hombre" más el vector de "mujer" da un resultado cercano al vector de "reina" — las relaciones semánticas quedan "codificadas" como relaciones geométricas entre vectores.

Word2Vec

El método más conocido para generar embeddings. Incluye dos variantes:

- **Skip-gram:** predice el CONTEXTO dada una palabra central. Funciona bien con palabras infrecuentes y da mejor representación de relaciones semánticas.
- **CBOW (Continuous Bag of Words):** predice la palabra CENTRAL dado su contexto. Es más rápido de entrenar y da mejor representación de palabras frecuentes.

Ambos son modelos **auto-supervisados** (no necesitan etiquetas manuales — la propia estructura del texto genera los pares de entrenamiento "palabra central / contexto").

Cómo se entrenan (mecanismo resumido)

1. Se usan dos matrices de embeddings: una para las palabras "centrales" y otra para las palabras de "contexto"
2. Se transforman las palabras/tokens en sus vectores (usando índices en esas matrices)
3. Se calcula la similitud entre el vector central y cada vector de contexto
4. Se calcula la función de pérdida (sigmoidea/logística)
5. Se actualizan los vectores centrales según el error

Al final del entrenamiento, se usa la matriz de vectores centrales como los embeddings finales, y la matriz de contextos se descarta.

Muestreo negativo (Negative Sampling)

En vez de comparar cada palabra central contra TODO el vocabulario (algo muy costoso computacionalmente), se aprende una decisión binaria simple:

- Pares reales (palabra central + su contexto real) → etiqueta 1
- Pares "inventados" (palabra central + una palabra al azar que NO es su vecina) → etiqueta 0

Esto evita el overfitting y acelera muchísimo el entrenamiento.

Subsampling / Submuestreo

Durante el armado de pares de entrenamiento, se descartan al azar palabras muy frecuentes (como "el", "de", "y"), ya que aportan poca información semántica y generan demasiados ejemplos, encareciendo el proceso. Esto acelera el entrenamiento, le da más protagonismo a las palabras menos frecuentes, y mejora la calidad general de los embeddings.

¿Para qué sirven los embeddings?

Son la base de entrada de las arquitecturas modernas de Deep Learning para NLP (incluyendo RNN, LSTM, y arquitecturas más avanzadas): en vez de representar palabras como un vector disperso tipo Bag of Words (donde cada palabra es una dimensión separada, sin relación entre sí), los embeddings capturan relaciones semánticas en un espacio denso de mucha menor dimensión.

10. REDES DE APRENDIZAJE PROFUNDO — conceptos generales

¿Por qué tardaron 50 años?

Dos problemas principales impedían entrenar redes profundas:

1. **Tiempo de entrenamiento:** sin GPUs modernas, podía tardar meses

2. **Desvanecimiento del gradiente:** al propagar el error hacia atrás por muchas capas, el gradiente se vuelve cada vez más pequeño hasta desaparecer, y las primeras capas no se actualizan

Todo cambió en 2006-2007 con papers de Hinton, LeCun y Bengio.

Restricted Boltzmann Machines (RBM)

Propuestas por Geoff Hinton como solución al problema de las redes profundas.

Arquitectura: Dos capas (visible y oculta), sin conexiones internas dentro de cada capa pero sí entre una y la otra, como un grafo bipartito.

Cómo funcionan:

1. Ejecutar una entrada en sentido directo (forward)
2. Hacer la ejecución en sentido inverso (backward)
3. Comparar usando KL Divergence y ajustar pesos y bias hasta que las salidas de la red invertida coinciden con las entradas

KL Divergence (Divergencia de Kullback-Leibler): Es la métrica que usan las RBM para el entrenamiento. Indica cuánta información perdemos al aproximar una distribución P (la real, los datos de entrada) con una distribución Q (nuestro modelo/aproximación). Cuanto menor sea la KL Divergence, mejor está reconstruyendo la red.

No usan backpropagation para el pre-entrenamiento. Son la base de las Deep Belief Nets.

Deep Belief Nets (DBN)

Son como un perceptrón multicapa, pero con un método de entrenamiento completamente diferente.

Entrenamiento en dos fases:

1. **Pre-entrenamiento no supervisado (greedy layer-wise):** Se entrena cada capa como una RBM, de abajo hacia arriba, capa por capa. Cada capa aprende el input entero. Al final, la red aprendió a detectar patrones, pero no sabe qué significan.
2. **Fine-tuning supervisado:** Con una cantidad pequeña de datos etiquetados, se ajustan los pesos con backpropagation tradicional.

Autoencoders

Red neuronal que aprende a producir a la salida exactamente la misma información que recibe a la entrada. La entrada y la salida tienen el mismo número de neuronas.

¿Para qué sirve si la salida es igual a la entrada? La clave está en la capa oculta del medio, que tiene MENOS neuronas. Eso obliga a la red a comprimir la información (como un cuello de botella).

Usos:

- Compresión / reducción de dimensionalidad (ej: 784 píxeles → 30 números)
- Eliminación de ruido (denoising autoencoder)
- Deep Fakes (ver abajo)

Entrenamiento: Se pueden entrenar con backpropagation, usando una métrica de "Loss" que mide cuánta información se perdió al reconstruir.

Deep Fakes con Autoencoders

1. Se entrena un autoencoder para la cara 1 (encoder compartido + decoder 1)
 2. Se entrena un autoencoder para la cara 2 (encoder compartido + decoder 2)
 3. "Trucazo": se pasa la cara 1 por el encoder compartido pero se usa el decoder de la cara 2 → genera la cara 1 con las características de la cara 2
-

11. GAN (Generative Adversarial Networks)

Redes generativas adversarias: dos redes compiten entre sí. Son redes **NO supervisadas** — no necesitan datos etiquetados.

Componentes:

- **Generador:** Crea datos falsos (ej: imágenes) a partir de ruido aleatorio
- **Discriminador:** Intenta distinguir entre datos reales y datos generados

Entrenamiento adversarial:

1. El generador crea una imagen falsa
2. El discriminador recibe imágenes reales y falsas, e intenta clasificarlas
3. El generador se actualiza para engañar mejor al discriminador
4. El discriminador se actualiza para detectar mejor las falsificaciones
5. Se repite hasta que el generador produce datos tan buenos que el discriminador no puede distinguirlos (equilibrio ~50%)

Usos: Generación de imágenes realistas, traducción de imagen a imagen, y creación de deep fakes.

12. HIPERPARÁMETROS DE UNA RED NEURONAL

Son parámetros que se configuran ANTES del entrenamiento (no se aprenden):

- **Learning rate (tasa de aprendizaje):** El más importante. Indica qué tan rápido se descende en la función de costo. Muy alto → oscila y no converge. Muy bajo → tarda demasiado.

- **Número de capas:** Más capas = más capacidad pero más riesgo de overfitting y desvanecimiento del gradiente.
 - **Número de neuronas por capa:** Similar al anterior. Más neuronas = más capacidad.
 - **Batch size:** Cuántos ejemplos se procesan antes de actualizar los pesos. Batch grande = más estable pero usa más memoria.
 - **Epochs:** Cuántas veces se pasa todo el dataset por la red. Más epochs = mejor hasta cierto punto, después overfitting.
 - **Optimizador:** SGD, Adam, RMSProp, etc.
 - **Función de activación:** ReLU, Sigmoide, Softmax, etc.
-

13. BATCHES Y EPOCHS

- **Epoch:** Una pasada completa por TODO el dataset de entrenamiento
- **Batch:** Un subconjunto del dataset que se procesa junto antes de actualizar los pesos
- **Iteración:** Cada vez que se procesa un batch

Ejemplo: si tengo 1000 datos y un batch size de 100, necesito 10 iteraciones para completar 1 epoch.

14. ESCALAMIENTO / NORMALIZACIÓN PARA REDES

Las redes neuronales son muy sensibles a las escalas de los datos de entrada. Si un feature tiene valores de 0-1 y otro de 0-1000000, la red va a dar mucho más peso al segundo.

Tipos de escalamiento:

- **Min-Max:** escala a rango $[0,1]$
- **Estandarización (Z-score):** escala a media=0, desvío=1

Se escala usando los parámetros del set de entrenamiento. Los mismos parámetros se aplican al set de test (NO se recalculan con test).

PREGUNTAS DE PRÁCTICA (formato final real)

Pregunta 1 (Desarrollo)

Explique brevemente: ¿Para qué se utiliza el método backpropagation? ¿Qué tipo de método es, es decir, qué recurso matemático utiliza?

Pregunta 2 (V/F con justificación)

Indique si las siguientes afirmaciones son verdaderas o falsas:

- a) Dropout es un método de regularización que evita el sobreajuste.
- b) La regularización L1/L2 nos evita tener que entrenar la red con backpropagation.
- c) La función Softmax permite ponderar la salida como probabilidad de la clasificación realizada.
- d) El perceptrón simple puede resolver cualquier problema de clasificación.

Pregunta 3 (Desarrollo)

Dibuje de forma sintética el esquema básico de una red convolucional, indicando capas, el orden de estas y las funciones de activación.

Pregunta 4 (V/F con justificación)

- a) Habitualmente Adam suele ser más rápido que RMSProp y que Momentum como optimizador.
- b) Para entrenar una red con backpropagation, las funciones de activación deben ser derivables.
- c) Backpropagation es un método alternativo al descenso por gradiente.
- d) Las redes SOM (Self-Organizing Maps) son no supervisadas, por lo que no requieren datos etiquetados.

Pregunta 5 (Desarrollo)

Explique los 3 tipos de estructuras que puede tener una red recurrente. Dé un ejemplo para cada una. ¿Cuál es el principal problema de una red recurrente?

Pregunta 6 (Desarrollo)

Explique brevemente los métodos de regularización para redes neuronales.

Pregunta 7 (Perceptrón)

Dada la siguiente tabla para una compuerta AND, resuelva la primera iteración del perceptrón con pesos iniciales $w_1=0.3$, $w_2=0.2$, $b=-1$ y tasa de aprendizaje $\alpha=0.2$. Calcule la salida con la función escalón.

RESPUESTAS MODELO

Respuesta 1

El método de backpropagation se utiliza para entrenar redes neuronales, ajustando los pesos de la red para minimizar el error entre la salida predicha y la real. Es un método de optimización supervisado. Utiliza el cálculo de derivadas parciales y la regla de la cadena para propagar el error desde la capa de salida hacia las capas anteriores y actualizar los pesos de manera eficiente.

Respuesta 2

- a) **VERDADERO.** Dropout apaga neuronas al azar durante el entrenamiento, forzando a la red a aprender de forma más robusta y a no depender de unas pocas neuronas, previniendo el sobreajuste.
- b) **FALSO.** L1 y L2 modifican la función de pérdida para penalizar pesos grandes, pero backpropagation sigue siendo el mecanismo para calcular los gradientes y actualizar los pesos. No se reemplazan, trabajan juntos.
- c) **VERDADERO.** Softmax transforma las salidas numéricas de la red (logits) en una distribución de probabilidad donde cada valor está entre 0 y 1 y todos suman 1, permitiendo interpretar la salida como la confianza del modelo en cada clase.
- d) **FALSO.** El perceptrón simple solo puede resolver problemas linealmente separables. No puede resolver problemas como XOR, donde las clases no se pueden separar con una línea recta.

Respuesta 3

Esquema CNN:

```
Entrada (imagen)
→ Capa Convolucional (aplica filtros para detectar patrones)
→ Capa ReLU (función de activación, introduce no-linealidad)
→ Capa Pooling (reduce dimensión espacial, ej: Max Pooling)
→ [Se repiten Conv + ReLU + Pooling varias veces]
→ Capa Fully Connected (clasifica usando todas las features extraídas)
→ Salida (clase predicha)
```

Las capas convolucionales extraen características de la imagen mediante filtros. ReLU aplica la función $\max(0, x)$. Pooling reduce la dimensión. La capa FC al final clasifica. Se usan múltiples bloques Conv+ReLU+Pooling pero generalmente una sola capa FC.

Respuesta 4

- a) **VERDADERO.** Adam combina Momentum y RMSProp, adaptando la tasa de aprendizaje por parámetro y usando impulso de actualizaciones pasadas, lo que generalmente lo hace converger más rápido.
- b) **VERDADERO.** Backpropagation calcula gradientes (derivadas parciales) para actualizar los pesos, por lo que las funciones de activación deben ser derivables.
- c) **FALSO.** Backpropagation no es alternativo al descenso por gradiente, sino una forma de aplicar el descenso por gradiente a redes neuronales.
- d) **VERDADERO.** Las redes SOM (Self-Organizing Maps) son no supervisadas, solo necesitan un conjunto de datos representativo, no datos etiquetados.

Respuesta 5

Los 3 tipos de estructuras de una RNN son:

1. **Secuencia a secuencia (many-to-many):** La entrada es una secuencia y la salida también. Ejemplo: predicción de precios de acciones (se ingresan precios de los últimos N días, la salida son precios futuros).
2. **Secuencia a valor singular (many-to-one):** La entrada es una secuencia y la salida es un único valor. Ejemplo: análisis de sentimiento de una reseña (secuencia de palabras → clasificación positivo/negativo).
3. **Valor singular a secuencia (one-to-many):** La entrada es un dato único y la salida es una secuencia. Ejemplo: generación de subtítulos para imágenes (se pasa una imagen, la red genera una secuencia de palabras que la describe).

El principal problema de las RNN es el desvanecimiento del gradiente, que es exponencialmente peor que en redes feedforward porque una capa recurrente con 100 pasos temporales equivale a entrenar una red de 100 capas. Soluciones: LSTM, GRU (gating), gradient clipping.

Respuesta 6

Los métodos de regularización para redes neuronales son:

- **Regularización L1 y L2:** Penalizan el valor de los pesos de la red, evitando que se le dé más relevancia a una característica por sobre otra. Puede hacer que el modelo generalice mejor.
- **Dropout:** Evita codependencias en las conexiones de la red. "Apaga" neuronas aleatoriamente durante el entrenamiento, haciendo que la red no dependa de unas pocas neuronas.
- **Early Stopping:** Detiene el entrenamiento antes de que el error del conjunto de validación empiece a crecer, evitando el overfitting.
- **Data Augmentation:** Genera datos nuevos usando los existentes pero aplicándoles transformaciones (rotaciones, recortes, etc.). Útil especialmente con imágenes.